

Tutorial: Socket Programming in Java

Pravish Sainath

1. Introduction to Socket Programming

In today's interconnected world, applications rarely work in isolation. Whether it is a web browser fetching a page, a multiplayer game syncing player positions, or a chat application delivering messages, they all rely on network communication.

Socket programming is the foundational technology that allows two software applications to talk to each other over a network. A **socket** is simply one endpoint of a two-way communication link between two programs running on a network.

In this tutorial, we will focus on the **Client/Server architecture** using the Transmission Control Protocol (TCP). TCP ensures that data sent from one end is received reliably and in the correct order at the other end.

2. Core Concepts: The Postal Service Analogy

Before writing code, you must understand how data finds its way across a network. We can map this directly to how the postal service works.

- **IP Address (The Street Address):** Every machine connected to a network has a unique Internet Protocol (IP) address. It identifies the specific computer you want to reach (e.g., 192.168.1.5 or `localhost` / 127.0.0.1 for your own machine).
- **Port Number (The Apartment Number):** A single computer runs many applications simultaneously (a web browser, an email client, a database). The port number (ranging from 0 to 65535) identifies exactly *which* application on that computer the data is meant for.
- **Socket (The Mailbox):** The socket is the software interface that the application uses to send and receive data. It binds an IP address and a Port number together.

3. The Client-Server Lifecycle

In a Client/Server model, the responsibilities are divided:

1. **The Server** is a program that runs continuously, waiting for incoming connections on a specific port. It “listens” for requests and serves them.
2. **The Client** is a program that initiates contact with the server to request a service or exchange data.

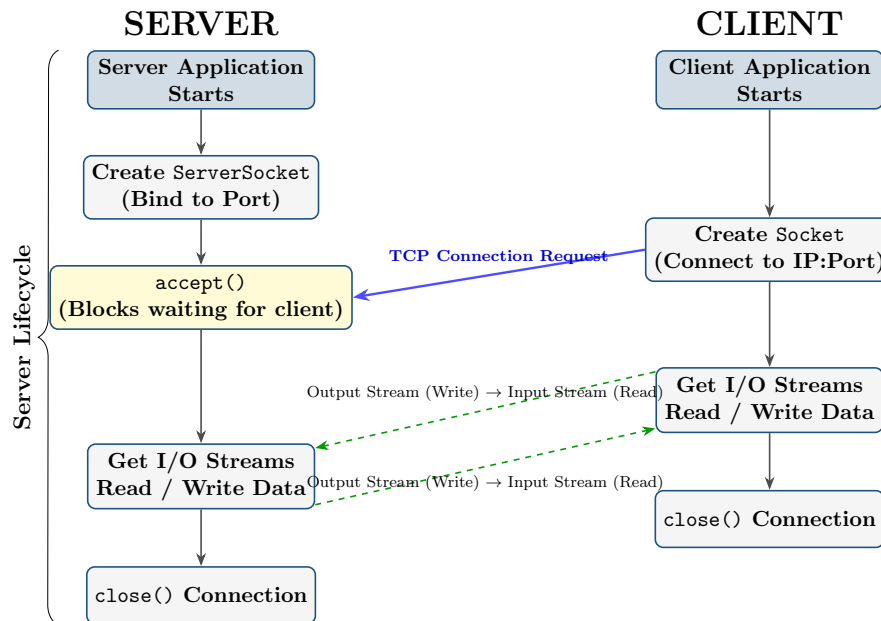


Figure 1: The typical lifecycle of a Java TCP Client/Server application. The Server opens a port and waits via `accept()`. The Client connects using the Server’s IP and Port. Once connected, both sides extract Input and Output streams to send and receive data before cleanly closing the connection.

Real-World Example: Web Browsing

To understand this lifecycle, consider what happens when you visit a website. Your web browser (the **Client**) initiates a connection to a remote web server (e.g., Google’s servers).

- The browser uses its **OutputStream** to **write** an HTTP GET request (e.g., “Give me the homepage”).
- The server uses its **InputStream** to **read** this request.
- After processing, the server takes the HTML code of the homepage and uses its **OutputStream** to **write** the data back.
- Finally, your browser’s **InputStream** **reads** this HTML data, renders the page on your screen, and the connection is closed.

4. Essential Java Networking Classes

The Java API makes network programming incredibly straightforward via the `java.net` and `java.io` packages.

4.1 The Server Classes

- `ServerSocket(int port)`: Used by the server to open a port and listen for client requests.
- `accept()`: A method of `ServerSocket`. When called, it *blocks* (pauses execution) until a client connects. Once a client connects, it returns a regular `Socket` object representing the established connection.

4.2 The Client Classes

- `Socket(String host, int port)`: Used by the client to initiate a connection. It automatically attempts to connect to the specified IP (host) and port.

4.3 I/O Streams (The Pipes)

Once a `Socket` connects, you need a way to pass data through it. Java handles this using streams. Think of streams as pipes connecting the client and server.

- `getInputStream()`: Receives data from the network.
- `getOutputStream()`: Sends data to the network.

Wrapping Streams for Text: Raw streams handle pure bytes (0s and 1s). Because we usually want to send text (Strings), we wrap these byte streams in character streams:

```
// To READ text easily (line by line):
BufferedReader in = new BufferedReader(new InputStreamReader(socket.
    getInputStream()));

// To WRITE text easily (and auto-flush):
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

5. Example 1: One-Way Communication

Let's build a simple program where a Client connects to a Server, sends a single greeting string, and disconnects. The Server will print the greeting to the console.

Real-World Example: Imagine a simple **Web Analytics Tracker** or an **Error Reporter** within an app. When your application (the **Client**) encounters a crash, it immediately connects

to a central logging **Server**, pushes a single error report string via one-way communication, and then disconnects to resume functioning. The server just listens, receives the crash report, logs it to a file, and closes its end of the connection.

5.1 The Simple Server

```
import java.io.*;
import java.net.*;

public class SimpleServer {
    public static void main(String[] args) {
        int port = 5000; // Choose an arbitrary port above 1024

        // 1. Create ServerSocket in a try-with-resources block
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println( Server is listening on port  + port
        );

            // 2. Wait for a client to connect (this blocks)
            Socket clientSocket = serverSocket.accept();
            System.out.println( New client connected! );

            // 3. Extract the input stream to read data from the
            client
            InputStream input = clientSocket.getInputStream();
            BufferedReader reader = new BufferedReader(new
            InputStreamReader(input));

            // 4. Read the message sent by the client
            String message = reader.readLine();
            System.out.println( Message from client:  + message);

            // 5. Clean up
            clientSocket.close();
            System.out.println( Server shut down. );

        } catch (IOException ex) {
            System.out.println( Server exception:  + ex.getMessage
            ());
            ex.printStackTrace();
        }
    }
}
```

5.2 The Simple Client

```
import java.io.*;
import java.net.*;

public class SimpleClient {
    public static void main(String[] args) {
        String hostname = localhost ; // Server IP (localhost means
        this computer)
        int port = 5000;                // Must match the Server's
        port

        // 1. Connect to the server
        try (Socket socket = new Socket(hostname, port)) {

            // 2. Extract the output stream to send data to the
            server
            OutputStream output = socket.getOutputStream();
            // 'true' enables auto-flushing so data sends
            immediately
            PrintWriter writer = new PrintWriter(output, true);

            // 3. Send a message
            writer.println( Hello, Server! This is my first socket
            connection. );
            System.out.println( Message sent to server. );

        } catch (UnknownHostException ex) {
            System.out.println( Server not found:  + ex.getMessage
            ());
        } catch (IOException ex) {
            System.out.println( I/O error:  + ex.getMessage());
        }
    }
}
```

How to run this: 1. Compile both files: `javac SimpleServer.java SimpleClient.java`
2. Open two terminal windows.
3. In Terminal 1, run the server: `java SimpleServer`
4. In Terminal 2, run the client: `java SimpleClient`

>_ Output

```
// Terminal 1 (Server Output) Server is listening on port 5000 New client
connected! Message from client: Hello, Server! This is my first socket
connection. Server shut down.
// Terminal 2 (Client Output) Message sent to server.
```

6. Example 2: Two-Way Communication (Echo Server)

One-way communication is rarely enough. Usually, a client sends a request, and the server sends a response. Let's create an **Echo Server**.

An Echo Server receives text from the client, prefixes it with “Echo: ”, and sends it right back. This loop continues until the client types “bye”.

Real-World Example: This two-way communication mimics a basic **Chatbot** or a **Dictionary Lookup Service**. You send a word (request), the server processes it (finds the definition), and sends the meaning back to you (response). The conversation stays open until you explicitly end it.

```

import java.io.*;
import java.net.*;

public class EchoServer {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(6000)) {
            System.out.println( Echo Server listening on port
6000... );

            Socket socket = serverSocket.accept(); // Wait for
client
            System.out.println( Client connected. );

            // Get both Input and Output streams
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream
(), true);

            String text;
            // Read lines continually until the client disconnects
or says bye
            while ((text = in.readLine()) != null) {
                System.out.println( Received:  + text);

                // Send response back to client
                out.println( Echo:  + text);

                if (text.equalsIgnoreCase( bye )) {
                    break;
                }
            }
            socket.close();
            System.out.println( Connection closed. );
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}

```

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class EchoClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket( localhost , 6000)) {

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream
(), true);

            // Scanner to read input from the user's console
            Scanner console = new Scanner(System.in);
            String userInput;

            System.out.println( Connected to Echo Server. Type 'bye'
to quit. );

            do {
                System.out.print( Enter text: );
                userInput = console.nextLine();

                // Send to server
                out.println(userInput);

                // Read response from server
                String response = in.readLine();
                System.out.println(response);

            } while (!userInput.equalsIgnoreCase( bye ));

        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}

```

7. Example 3: Handling Multiple Clients (Multi-threading)

The `EchoServer` in Example 2 has a massive flaw: **It can only handle one client at a time.** Because the server gets trapped in the `while` loop reading from Client A, if Client B tries to connect, Client B must wait until Client A types “bye”.

In the real world (like a web server or chat room), servers must handle thousands of clients concurrently. To achieve this in Java, we use **Threads**.

The pattern is: 1. The main server thread runs in an infinite loop, calling `accept()`. 2. When a client connects, `accept()` returns a `Socket`. 3. The server immediately hands this `Socket` to a *new Thread* (a Client Handler). 4. The main thread goes right back to `accept()` to wait for the next client.

Real-World Example: Think of a **Multiplayer Game Server** or a busy **Restaurant**. The main `ServerSocket` is the host at the front door. When you arrive, the host doesn’t serve you directly; they assign you a dedicated waiter (a new **Thread**). The host then immediately returns to the door to welcome the next guest. This way, dozens of tables (clients) can order food simultaneously without waiting for each other.

7.1 The Multi-threaded Server

```
import java.io.*;
import java.net.*;

public class MultiThreadedServer {
    public static void main(String[] args) {
        int port = 7000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println( "Multi-threaded Server listening on
port    + port);

            // Infinite loop to continuously accept new clients
            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println( "New client connected:    + socket
.getInetAddress());

                // Hand the socket over to a new Thread
                ClientHandler handler = new ClientHandler(socket);
                new Thread(handler).start();
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

```

// (Continuing in the same file...)
// Runnable class to handle individual client communication
class ClientHandler implements Runnable {
    private Socket socket;

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream
(), true);

            out.println( Welcome to the Multi-threaded Server! );

            String text;
            while ((text = in.readLine()) != null) {
                System.out.println( Received:    + text);
                out.println( Server processed:    + text);
                if (text.equalsIgnoreCase( quit )) break;
            }

            System.out.println( Client disconnected. );
            socket.close();
        } catch (IOException ex) {
            System.out.println( Client error:    + ex.getMessage());
        }
    }
}

```

*Note: You can use multiple instances of the **EchoClient** (changing the port to 7000) to connect to this server simultaneously. You will notice that both clients get responses immediately, proving that they are being handled on separate threads.*

8. Key Points to Remember

Key Points to Remember

- A **Socket** requires an **IP Address** (to find the machine) and a **Port** (to find the application).
- The server uses **ServerSocket** to open a port and **accept()** to wait for incoming connections.
- The client uses **Socket** to actively connect to a server.
- Data is transmitted via **InputStreams** (Reading) and **OutputStreams** (Writing).
- Wrap raw streams in **BufferedReader** and **PrintWriter** when exchanging text to easily read and write data line-by-line.
- Pass **true** to **PrintWriter** for **auto-flush**, otherwise, your messages might get stuck in a memory buffer and never traverse the network.
- A basic server is **blocking**. To handle multiple clients simultaneously, you must spawn a new **Thread** for every accepted client socket.
- Always close your sockets and streams in a **finally** block or use a **try-with-resources** statement to prevent memory and port leaks.

9. Practice Exercise: A Simple Calculator Server

Test your knowledge by modifying the Echo Server into a **Calculator Server**.

Requirements:

1. Create a server that accepts multiple clients concurrently.
2. The client should prompt the user to enter a math problem (e.g., “5 + 10” or “20 * 3”).
3. The client sends this string to the server.
4. The server reads the string, parses the numbers and the operator, performs the calculation, and sends the result string back (e.g., “Result: 15”).
5. If the user sends an invalid format, the server should reply with “Error: Invalid format”.

*Hint: Use **String.split(" ")** on the server side to separate the operands and the operator.*

A. Socket API Quick Reference

This table provides a summary of the most commonly used methods in Java socket programming.

Class / Method	Short Description
— <i>ServerSocket Class (java.net.ServerSocket)</i> —	
ServerSocket(int port) Socket accept() void close()	Creates a server socket, bound to the specified port. Listens for a connection to be made to this socket and accepts it. The method blocks until a connection is made. Closes this socket.
— <i>Socket Class (java.net.Socket)</i> —	
Socket(String host, int port) InputStream getInputStream() OutputStream getOutputStream() InetAddress getAddress() void close()	Creates a stream socket and connects it to the specified port number on the named host. Returns an input stream for reading bytes from this socket. Returns an output stream for writing bytes to this socket. Returns the address to which the socket is connected. Closes this socket, also closing its associated Input/Output streams.
— <i>Stream Wrappers (java.io.*)</i> —	
new BufferedReader(...) new InputStreamReader(...) new PrintWriter(..., true) String readLine() void println(String s)	Wraps a character stream to buffer the input, allowing for the reading of entire lines of text via <code>readLine()</code> . A bridge from byte streams to character streams. Wraps an output stream to print formatted text representations of objects. The second argument enables auto-flushing. (BufferedReader) Reads a line of text. Returns null if the end of the stream has been reached. (PrintWriter) Prints a String and then terminates the line.

Table 1: Summary of Common Java Networking and I/O Methods.

References and Further Reading

Here is a curated list of official documentation and high-quality tutorials for deepening your understanding of Java Network Programming.

Oracle’s Official Java Networking Tutorial The authoritative guide from Oracle covering all aspects of custom networking in Java, including Sockets, Datagrams, and URLs.
docs.oracle.com/javase/tutorial/networking/index.html

Baeldung: A Guide to Java Sockets A well-structured guide providing clear, step-by-step examples of establishing socket connections, sending data, and implementing multi-threading.
baeldung.com/a-guide-to-java-sockets

GeeksforGeeks: Socket Programming in Java A beginner-friendly tutorial breaking down the Client-Server architecture with simple, copy-pasteable examples for immediate testing.
[geeksforgeeks.org/socket-programming-in-java](https://www.geeksforgeeks.org/socket-programming-in-java)

Official Javadoc for java.net The official API documentation for all classes and interfaces in the networking package (e.g., `Socket`, `ServerSocket`).
docs.oracle.com/.../java/net/package-summary.html

Official Javadoc for java.io The official API documentation for stream wrappers used in network communications (e.g., `BufferedReader`, `PrintWriter`).
docs.oracle.com/.../java/io/package-summary.html

DigitalOcean: Java Socket Programming Examples Excellent supplementary examples demonstrating concurrent client handling and file transfer over TCP sockets.
[digitalocean.com/community/tutorials/java-socket-programming](https://www.digitalocean.com/community/tutorials/java-socket-programming)